Robotica (2025), **43**, pp. 4519–4537
doi:10.1017/S0263574725102993

CAMBRIDGE UNIVERSITY PRESS

RESEARCH ARTICLE

Deep reinforcement learning-based obstacle avoidance motion planning for redundant manipulator considering the actual shape of obstacles

Qing Yang[1], Ju Chen[1], Yi Zhang[1], Liang Ge[2] and Yun Xiao Wang[1]

[1]School of Electrical Engineering and Information, Southwest Petroleum University, Chengdu, China
[2]School of Mechanical and Electrical Engineering, Southwest Petroleum University, Chengdu, China
Corresponding author: Ju Chen; Email: wy_cj2024@163.com

Received: 21 May 2025; **Revised:** 31 July 2025; **Accepted:** 19 November 2025; **First published online:** 18 December 2025

Keywords: deep reinforcement learning; distance algorithm; motion planning; composite reward function

Abstract

Although deep reinforcement learning (DRL) techniques have been extensively studied in the field of robotic manipulators, there is limited research on directly mapping the output of policy functions to the joint space of manipulators. This paper proposes a motion planning scheme for redundant manipulators to avoid obstacles based on DRL, considering the actual shapes of obstacles in the environment. This scheme not only accomplishes the path planning task for the end-effector but also enables autonomous obstacle avoidance while obtaining the joint trajectories of the manipulator. First, a reinforcement learning framework based on the joint space is proposed. This framework uses the joint accelerations of the manipulator to calculate the Cartesian coordinates of the end-effector through forward kinematics, thereby performing end-to-end path planning for the end-effector. Second, the distance between all the linkages of the manipulator and irregular obstacles is calculated in real time based on the Gilbert–Johnson–Keerthi distance algorithm. The reward function containing joint acceleration is constructed with this distance to realize the obstacle avoidance task of the redundant manipulator. Finally, simulations and physical experiments were conducted on a 7-degree-of-freedom manipulator, demonstrating that the proposed scheme can generate efficient and collision-free trajectories in environments with irregular obstacles, effectively avoiding collisions.

1. Introduction

In recent years, redundant manipulators have been playing an increasingly important role in various fields, including industry [1], space exploration [2], and others [3, 4]. In these scenarios, the most common task for redundant manipulators is to move the end-effector from the starting position to the target position, such as in assembly tasks and handling tasks. Researchers have divided the operation process of redundant manipulators into two research parts. The first part is path planning, which involves planning a feasible point-to-point path in the environment to enable the end-effector to reach the target point. The second part is trajectory planning, which involves converting the path points of the end-effector into joint information with a time sequence for the redundant manipulator. Moreover, due to the complex environment and the structural limitations of the manipulator itself, other subtasks need to be performed simultaneously during path and trajectory planning. For example, obstacle avoidance [5, 6] and joint physical limit avoidance [7, 8].

Traditional motion planning methods for redundant manipulators usually separate path planning and trajectory planning into a two-step inverse solution. The manipulator end-effector is abstracted as a point or sphere in path planning, and feasible paths present in the environment are searched using algorithms such as Rapidly-Exploring Random Tree (RRT) algorithm [9], A∗

© The Author(s), 2025. Published by Cambridge University Press.

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993

algorithm, artificial potential field method [10, 11], and metaheuristic [12]. In trajectory planning, researchers usually use approaches such as the gradient projection method, the extended Jacobi method, and the optimization function solution method [13–15]. Trajectory planning involves solving the inverse kinematics (IK) of the manipulator to obtain joint angles, angular velocities, and angular accelerations from the smoothed path points. Although the integration of the above algorithms has effectively addressed the motion planning task for manipulators, there are still some issues that cannot be ignored. First, both RRT and A∗ path planning algorithms require the preconstruction of an environment map, and they lack extendibility in dealing with unknown environments, and may even suffer from the curse of high dimensionality. Moreover, these path planning algorithms do not take into account the actual physical configuration of the manipulator. However, it is not only the end-effector that needs to avoid obstacles, but all the links of the manipulator must not come into contact with obstacles during motion. Second, solving the IK is difficult [16, 17]. Since the joint dimension of redundant manipulators is greater than the dimension of the environment, the calculation of the inverse of the Jacobian matrix is very complex and there are multiple solutions. Furthermore, due to the need to consider the links of the manipulator avoiding obstacles (especially irregular obstacles) and joint position constraints, the inverse kinematic solution may face no solution or singular solutions.

In recent years, research at the intersection of deep reinforcement learning (DRL) and robotics has provided new solutions for today's robotics [18, 19]. In robot path planning, DRL does not require the construction of an environment map [20, 21]. The agent learns to obtain the optimal policy output. Moreover, when facing high-dimensional environments, since the policy is approximated using neural networks, there is no risk of dimensionality explosion. In trajectory planning, DRL has been studied mainly in place of the inverse kinematic solution process [22]. The manipulator learns through continuous interaction with the environment to approximate the neural network function representing the IK, thereby avoiding the complex calculation of the Jacobian matrix and improving the efficiency of trajectory planning and the quality of the optimized solution [23, 24]. However, DRL also has problems such as slow training speed and non-convergence, mainly due to insufficient environment feature extraction and improper reward function settings [25, 26]. Moreover, the aforementioned reinforcement learning algorithms only improve the speed of motion planning and environmental adaptability, but do not change the motion planning method of first performing path planning and then trajectory planning. In recent years, Dai et al. [27] designed a motion planning scheme for manipulators in the joint space, combining artificial potential fields and the RRT algorithm, to avoid complex IK calculations and improve obstacle avoidance efficiency. Liu et al. [28] proposed a general framework based on DRL, where the output of the policy function is mapped to the joint space. This not only saves the process of solving IK for trajectory planning but also allows for timely replanning when the task scenario changes. Although refs. [27, 28] use different algorithms, the basic idea is the same: to use the joint space as the output of motion planning and employ the forward kinematics model instead of the IK model. This idea is different from the traditional two-step motion planning method and provides a new direction for solving motion planning problems. However, both algorithms employ the artificial potential field method for obstacle avoidance, abstracting the target point and obstacles as points that provide attractive and repulsive forces, which may lead to some incorrect solution sets. Moreover, both algorithms use joint angles as the output. To meet the requirements of motion planning, there may be abrupt changes in joint angles. However, in practical applications, such ideal abrupt changes do not exist because they would generate significant torques, which could damage the manipulator.

To address the aforementioned issues, this paper proposes a motion planning scheme for redundant manipulators to avoid obstacles based on a DRL algorithm, considering the actual shapes of obstacles. This scheme integrates the forward kinematics model of the manipulator into the DRL framework, which can not only solve the path planning task for the end-effector but also perform obstacle avoidance trajectory planning for all joints and links of the manipulator. The specific contributions are as follows:

1. A DRL algorithm scheme that uses joint accelerations as policy outputs is proposed. By employing multilevel physical limit avoidance for joints, it not only prevents the generation of large torques but also meets the motion planning task without IK solving.

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993

2. The Gilbert–Johnson–Keerthi (GJK) distance algorithm is integrated into the DRL scheme to obtain the accurate real-time distance between the manipulator and irregular convex polyhedral obstacles, providing support for the construction of a composite reward function.
3. A composite reward function mainly constructed from joint accelerations and velocities is proposed. This function aims to achieve collision-free motion for both the end-effector and the manipulator links. It takes into account the position error between the end-effector and the target point, as well as the actual minimum distance between the links and the obstacles.

The remainder of this paper is organized as follows. Section 2 presents the overall framework of the motion planning. Section 3 introduces the design details of the motion planning scheme, including the establishment of the manipulator's kinematics model and the joint physical limit mechanism, the idea of the GJK algorithm, and the framework of the DRL algorithm. Section 4 describes the construction of the composite reward function. Section 5 provides the simulation and experimental verification and analysis. Section 6 summarizes the method and proposes future research directions.

2. Overall framework of motion planning

The overall motion planning scheme in this paper can be divided into three parts, including the kinematics modeling of the manipulator, the GJK distance algorithm, and the DRL algorithm. In Figure 1, first, the physical configuration is obtained from the kinematics model of the manipulator. Then, the links of the manipulator and the obstacles are abstracted as convex polyhedron, and the GJK distance algorithm is used to obtain the coordinates of the nearest point to the obstacle. Second, through the forward kinematics model and coordinate locking, the velocity and acceleration parameters of the point on the manipulator that is closest to the obstacle are obtained. These parameters are provided to the reward function in reinforcement learning, and the network parameters are updated through the learning mechanism. Finally, the updated reinforcement learning outputs the angular acceleration for the next state. After passing through the physical joint limit, the forward kinematics model is used to update the physical configuration of the manipulator. By integrating the above three steps, a closed loop of the motion planning scheme is formed. The pseudocode for the overall scheme is shown in Algorithm 1.

3. Design of the motion planning scheme

3.1. Construction of the manipulator kinematics model

The forward kinematics model of the manipulator can be expressed as

$$g_E = f(\theta) \tag{1}$$

where $g_E \in R^m$ and $\theta \in R^n$ are the Cartesian path of the end-effector and the joint angle between the links, respectively. $m > 0$ and $n > 0$ are the dimensions of the operational space and the degree of freedom (DOF) of the manipulator, respectively. When $n > m$, the DOF of the manipulator is greater than the dimensions of the operational space, allowing it to perform additional subtasks.

Taking the time derivative of (1) yields the velocity-level forward kinematics formula:

$$\dot{g}_E = J_E \dot{\theta} \tag{2}$$

where $\dot{g}_E \in R^m$ and $\dot{\theta} \in R^n$ are the velocities of the end-effector and the joint angular velocities between the links, respectively. $J_E \in R^{m \times n}$ is the Jacobian matrix of the end-effector, denoted as $J_E = \partial f(\theta)/\partial(\theta)$.

Taking the time derivative of (2) yields the acceleration-level forward kinematics formula:

$$\ddot{g}_E = \dot{J}_E \dot{\theta} + J_E \ddot{\theta} \tag{3}$$

where $\ddot{g}_E \in R^m$ and $\ddot{\theta} \in R^n$ are the accelerations of the end-effector and the joint angular accelerations between the links, respectively. $\dot{J}_E \in R^{m \times n}$ is the derivative of $J_E$with respect to time t.

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993

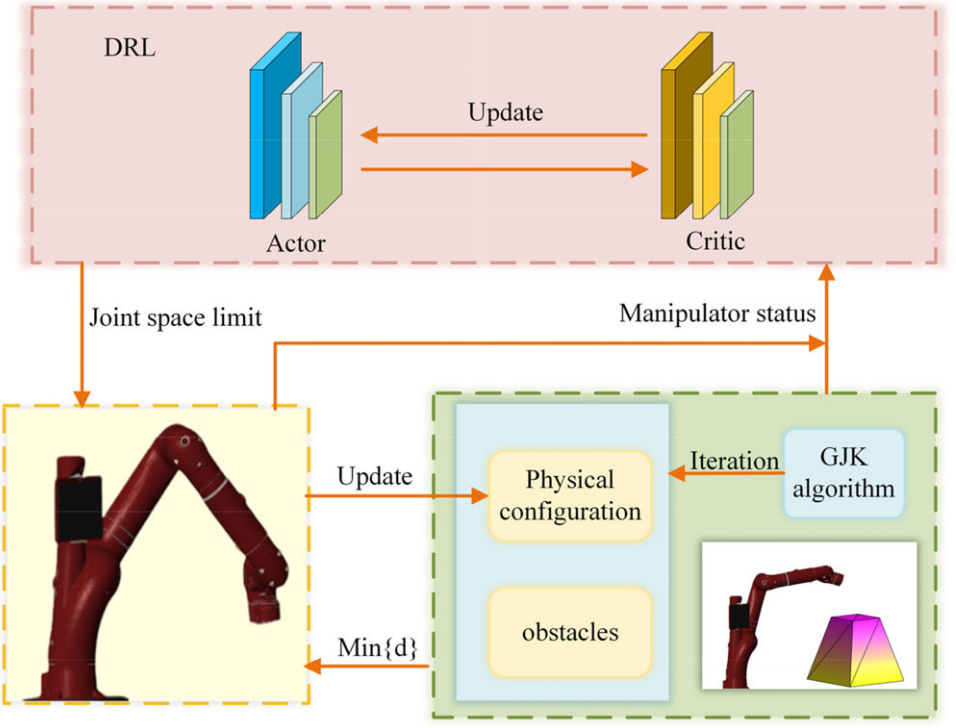


Figure 1. *Overall framework of motion planning.*

In practical engineering applications, to ensure the safety and reliability of the manipulator during operation, the following physical limits for joint variables are defined:

$$\begin{aligned} \theta^{-} &\le \theta \le \theta^{+} \\ \dot{\theta}^{-} &\le \dot{\theta} \le \dot{\theta}^{+} \\ \ddot{\theta}^{-} &\le \ddot{\theta} \le \ddot{\theta}^{+} \end{aligned} \tag{4}$$

where $\theta^{\pm} \in R^n$, $\dot{\theta}^{\pm} \in R^n$ and $\ddot{\theta}^{\pm} \in R^n$ are the upper and lower limits of angle, angular velocity, and angular acceleration, respectively. The joint variables physical limits in (4) are unified as acceleration levels:

$$\zeta^{-} \le \ddot{\theta} \le \zeta^{+} \tag{5}$$

where $\zeta^{-} \in R^n$, $\zeta^{+} \in R^n$,denoted by

$$\begin{aligned} \zeta^{-} &= \max\left[\kappa_p\left(\mu\theta^{-} - \theta\right), \kappa_v\left(\dot{\theta}^{-} - \dot{\theta}\right), \ddot{\theta}^{-}\right] \\ \zeta^{+} &= \min\left[\kappa_p\left(\mu\theta^{+} - \theta\right), \kappa_v\left(\dot{\theta}^{+} - \dot{\theta}\right), \ddot{\theta}^{+}\right] \end{aligned} \tag{6}$$

where $\mu \in (0, 1)$ defines the critical regions $[\theta^{-}, \mu\theta^{-}]$ and $[\mu\theta^{+}, \theta^{+}]$ such that deceleration is performed by limiting the angular acceleration when the joint angle enters one of the critical regions. Furthermore, $\kappa_p > 0$ and $\kappa_v > 0$ determine the deceleration amplitude [16, 29].

Remark 1: Generally, the selection of a and b should ensure that the feasible region of joint acceleration c obtained from joint limits and joint velocity limits is not smaller than the feasible region obtained from the original joint acceleration.

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993

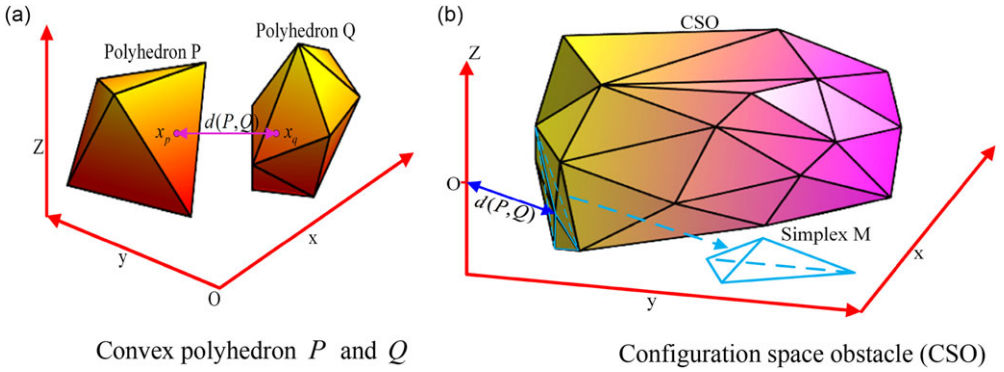


Figure 2. *Schematic of GJK algorithm distance calculation.*

3.2. GJK distance algorithm

Traditional distance algorithms in Cartesian coordinates do not effectively search for the shortest distance between the manipulator and obstacles in three-dimensional space, especially when the obstacles are not regular polyhedron or spheres. Therefore, researchers commonly use the artificial potential field method to establish the distance relationship between the manipulator and obstacles. However, the uniformly divergent repulsive or attractive forces generated by the artificial potential field method in space can lead to local optimal solutions in the solution space. That is, when the target point and the obstacle are very close, the manipulator may become trapped in a dilemma.

The GJK algorithm iteratively abstracts objects of any shape into convex polyhedral envelopes and accurately determines the closest points between these two convex bodies, thereby yielding their actual minimum distance [30, 31]. In Figure 2(a), there are two convex polyhedral objects P and Q, with their point sets in space being $Z_P \in R^3$ and $Z_Q \in R^3$. For these two convex polyhedrons, the nearest Euclidean distance $d(P, Q)$ can be expressed as

$$d(P, Q) = \min\left\{\left\|x_p - x_q\right\|, x_p \in Z_P, x_q \in Z_Q\right\} \tag{7}$$

In Figure 2(b), the Minkowski difference between two objects P and Q in the GJK distance algorithm leads to a configuration space obstacle (CSO):

$$CSO = P - Q = \left\{x_p - x_q, x_p \in Z_P, x_q \in Z_Q\right\} \tag{8}$$

By using the support function to selectively calculate the above Minkowski difference, an iterative generation of a simplex polyhedron is performed. The distance between P and Q is equal to the nearest distance from the simplex polyhedron M to the origin when the iteration termination condition is met. The point set of the simplex M is $Z_M \in R^3$. Hence, the existence:

$$d(P, Q) = \min\{\|m\|, m \in Z_M\} \tag{9}$$

At this point, two convex polyhedron P and Q collide if the simplex M contains the origin. It is worth noting that even if the objects being calculated are non-convex, the GJK distance algorithm can still identify the vertex coordinates that form the simplex during its iterative process.

3.3. DRL algorithm design

In DRL, the motion tasks of a robotic arm can be regarded as Markov decision processes (MDPs). The improvements proposed in this paper for manipulator motion planning are applicable to general reinforcement learning algorithms. However, algorithms such as Q-learning and SARSA have discrete action outputs, which may lead to the omission of correct manipulator configurations during the learning process. Although algorithms like Proximal Policy Optimization and Soft Actor-Critic can produce continuous actions, they have numerous hyperparameters that may introduce unknown effects on

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993

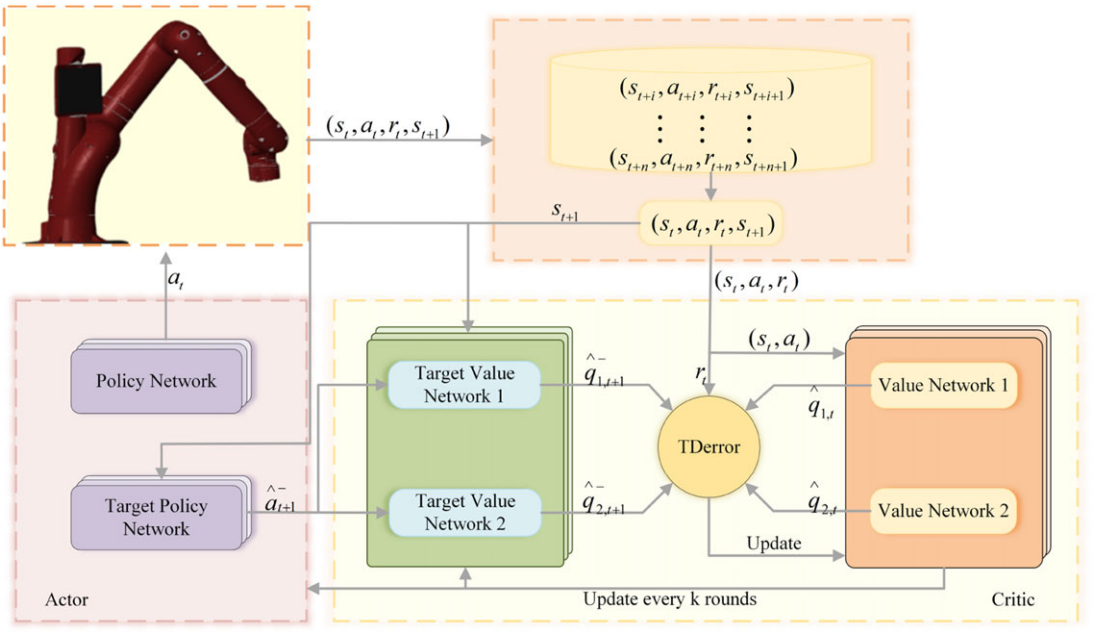


Figure 3. *Framework of the TD3 algorithm.*

the proposed improvements in this paper. Therefore, this paper employs the more stable TD3 (Twin Delayed Deep Deterministic Policy Gradient) algorithm to validate the motion planning scheme for the manipulator.

For the joint exploration actions driven by DRL, each pose is the IK solution of a complex model in traditional research [26, 32]. Therefore, in this paper, the joint angular acceleration between each linkage is proposed as the strategy network output a_t , defined as

$$a_t = \left[\ddot{\theta}_1, \ddot{\theta}_2, \cdots, \ddot{\theta}_i\right] \tag{10}$$

where $\ddot{\theta}_i$ is the angular acceleration of the ith joint. The state s_t is defined as

$$s_t = \left[C_0, C_1, E, A_{an}, A_{sp}, A_{ac}, D\right] \tag{11}$$

where $C_0 = (x_0, y_0, z_0)$ and $C_1 = (x_1, y_1, z_1)$ are the current positions of the end-effector and the target point based on the world coordinate system. $E = [\Delta x, \Delta y, \Delta z]$ represents the position difference between the end-effector and the target point. $A_{an} = [\theta_1, \theta_2, \cdots \theta_i]$ represents the joint angles of the manipulator. $A_{sp} = [\dot{\theta}_1, \dot{\theta}_2, \cdots \dot{\theta}_i]$ represents the joint angular velocities of the manipulator. $A_{ac} = a_t = [\ddot{\theta}_1, \ddot{\theta}_2, \cdots \ddot{\theta}_i]$ represents the joint angular accelerations of the manipulator. D represents the minimum distance between the manipulator and the obstacle.

In Figure 3, the TD3 algorithm, based on DDPG, uses truncated double Q-learning. By selecting the smaller of the two target value network estimates to limit the Q-value, the target $\hat{y}$ is calculated as follows:

$$\hat{y} = r_t + \gamma \min_{i=1,2} \left\{ q_i \left(s_{t+1}, \hat{a}^-_{t+1}, \omega^-_i \right) \right\} \tag{12}$$

where γ is the discount factor and ω^-_i is a parameter of the target value network. Where $\hat{a}^-_{t+1} = \mu(s_{t+1}; \delta^-) + \xi$, ξ denote the noise, each element of which is independently randomized from the truncated normal distribution $CN(0, \sigma^2, -\varphi, \varphi)$. Then, based on the prediction difference $\psi_i = \hat{q}_i - \hat{y}$

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993

Algorithm 1: The motion planning process of the manipulator

Initialize manipulator configuration $\theta, \dot{\theta}, \ddot{\theta}$.
Initialize actor network $\mu(s_t, \delta)$, critic network $q(s_t, a, \omega)$, respectively.
Initialize target network: $\delta^- \leftarrow \delta, \omega^- \leftarrow \omega$.
for each episode **do**
  Reset the environment.
  for each step **do**
    Select action by $a_t = \mu(s_t, \delta) + \xi$. Execute physical limit.
    Execute $g = f(a_t)$ by (1),
      Execute convex polyhedron modeling, D_{GJK} = iteration (CSO) by (9).
      Calculate the velocity and acceleration of the nearest point.
    Execute a_t and observe s_{t+1}, r by (27).
    Store (s_t, a_t, s_{t+1}) to memory.
    Update the network and target network by (12−15).
    if collision occurs or target point is reached **do**
      break.

between the value networks $\hat{q}_i$ and $\hat{y}$, the parameters of the value network are updated as follows:

$$\omega_i \leftarrow \omega_i - \alpha \psi_i \nabla_\omega q(s_t, a_t, \omega_i) \tag{13}$$

where $\nabla_\omega q(s_t, a_t, \omega_i)$ is the partial derivative of the parameters ω in the value network, and α is the learning rate of the value network.

To improve the stability of training, TD3 reduces the frequency of updating the strategy network and the target value network by updating the parameters every k round, updating the parameters δ in the strategy network:

$$\delta \leftarrow \delta + \beta \nabla_\delta \mu(s_t; \delta) \nabla_\omega q(s_j, \hat{a}_j; \omega) \tag{14}$$

where β is the learning rate of the policy network. $\nabla_\delta \mu(s_t; \delta)$ is the partial derivative of the parameters δ in the policy network update the target value network parameters ω_i^-, and the target strategy network parameters δ^- :

$$\begin{aligned} \delta^- &\leftarrow \delta + (1 - \phi)\delta^- \\ \omega_i^- &\leftarrow \omega_i + (1 - \phi)\omega_i^- \end{aligned} \tag{15}$$

where a denotes the smoothing factor when the network and the target network jointly update the parameters.

4. Composite reward function

In this section, a composite reward function composed of position rewards based on the GJK distance algorithm, obstacle avoidance rewards, and motion time rewards is proposed. This function can quickly enable the end-effector to reach the target position and allow all links of the manipulator to complete the obstacle avoidance task.

4.1. Position reward function for the end-effector

The joint angular accelerations obtained from the policy network are

$$\ddot{\theta}_t = \left[\ddot{\theta}_1, \ddot{\theta}_2, \cdots, \ddot{\theta}_i\right] \tag{16}$$

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993

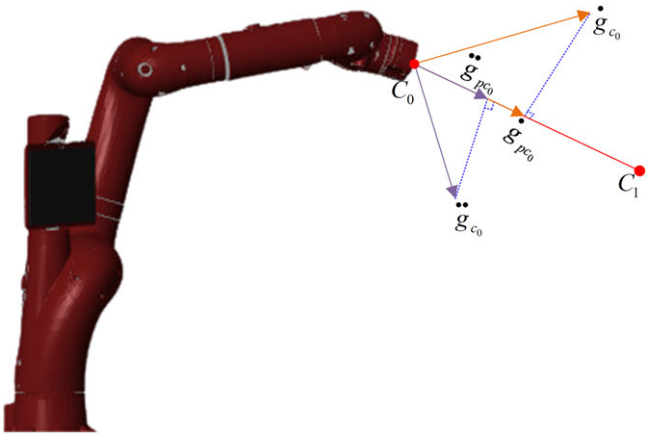


Figure 4. *Distance difference between the manipulator and the target point.*

After passing through the joint physical limit mechanism, the joint velocities and joint angles are calculated from the joint angular velocities:

$$\dot{\theta}_{t+1} = \dot{\theta}_t + \tau\ddot{\theta}_t \tag{17}$$

$$\theta_{t+1} = \dot{\theta}_t\tau + \ddot{\theta}_t\tau^2/2 \tag{18}$$

where τ is the discrete single-step movement time. In Figure 4, when the manipulator performs a motion task, the end-effector and the target point are denoted as C_0 and C_1, with coordinates (x_0, y_0, z_0) and (x_1, y_1, z_1), respectively, forming the vector $\overrightarrow{C_0C_1} = [x_1 - x_0, y_1 - y_0, z_1 - z_0]$. From (2) and (3), the velocity and acceleration of point C_0 can be obtained as

$$\dot{g}_{c_0} = J_{c_0}\dot{\theta} \qquad \ddot{g}_{c_0} = \dot{J}_{c_0}\dot{\theta} + J_{c_0}\ddot{\theta} \tag{19}$$

where $J_{c_0} \in R^{3\times n}$ is the Jacobi matrix at point C_0 and $\dot{J}_{c_0} \in R^{3\times n}$ is the derivative of J_{c_0} in time. By the projection theorem of vectors, the projections of the velocity and acceleration at point C_0 onto $\overrightarrow{C_0C_1}$ are

$$\dot{g}_{pc_0} = \dot{g}_{c_0} \cdot (\overrightarrow{C_0C_1}/\|\overrightarrow{C_0C_1}\|) \qquad \ddot{g}_{pc_0} = \ddot{g}_{c_0} \cdot (\overrightarrow{C_0C_1}/\|\overrightarrow{C_0C_1}\|) \tag{20}$$

where $a \cdot b = a_1b_1 + a_2b_2 + \cdots a_nb_n$ is the vector dot product.

By the distance formula, the distance that the manipulator advances on vector $\overrightarrow{C_0C_1}$ after time τ is

$$d_s = \dot{g}_{pc_0}\tau + \ddot{g}_{pc_0}\tau/2 \tag{21}$$

Design the position reward function of the end-effector as

$$r_p = \left(2/\left(1 + \exp\left(-k_p \times d_s\right)\right) - 1\right) \tag{22}$$

This function has high sensitivity at the origin and can quickly produce significant positive or negative output reward r_p for small changes in the input d_s, where k_p is a hyperparameter that can adjust the steepness of the function at the origin. Furthermore, the output of the function is limited to the range $(-1, 1)$. Even if the value of d_s is subject to unknown disturbances and violent fluctuations, the reward r_p will not exceed the specified value, which is beneficial for the smoothness and stability of training.

4.2. Obstacle avoidance task reward function

In Figure 5, when the manipulator performs the obstacle avoidance subtask, the points on the manipulator closest to the obstacle are C_2, C_3, with coordinates (x_2, y_2, z_2) and (x_3, y_3, z_3), respectively, forming the

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993

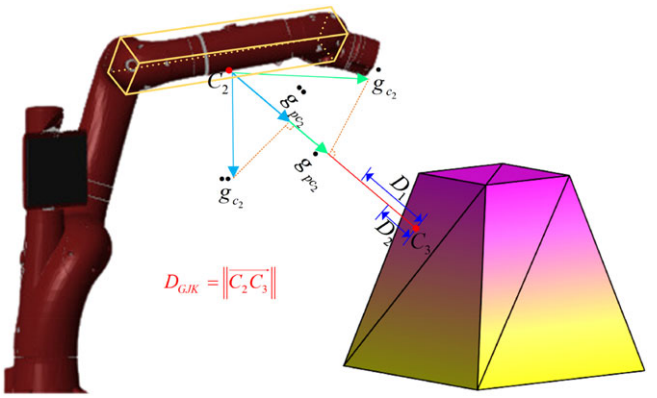


Figure 5. *Distance between the manipulator and the obstacle.*

vector $\overrightarrow{C_2C_3} = [x_2 - x_3, y_2 - y_3, z_2 - z_3]$. From (2) and (3), the velocity and acceleration at point C_2 can be obtained as

$$\dot{g}_{c_2} = J_{c_2}\dot{\theta}\ddot{g}_{c_2} = \dot{J}_{c_2}\dot{\theta} + J_{c_2}\ddot{\theta} \tag{23}$$

where $J_{c_2} \in R^{3\times n}$ is the Jacobi matrix at point C_2 and $\dot{J}_{c_2} \in R^{3\times n}$ is the derivative of J_{c_2} in time. By the projection theorem of vectors, the projections of the velocity and acceleration at point C_2 onto $\overrightarrow{C_2C_3}$ are

$$\dot{g}_{pc_2} = \dot{g}_{c_2} \cdot (\overrightarrow{C_2C_3}/\|\overrightarrow{C_2C_3}\|)\ddot{g}_{pc_2} = \ddot{g}_{c_2} \cdot (\overrightarrow{C_2C_3}/\|\overrightarrow{C_2C_3}\|) \tag{24}$$

Define three distances: D_{GJK}, D_1,andD_2. Model the links of the manipulator and the obstacles as convex polyhedron composed of points, then d is the real-time minimum distance between the manipulator and the obstacle detected by the GJK distance algorithm. D_1 is the safe distance defined for the manipulator to perform obstacle avoidance. When $D_{GJK} < D_1$, the distance d_o is obtained from the distance formula:

$$d_o = \dot{g}_{pc_2}\tau + \ddot{g}_{pc_2}\tau/2 \tag{25}$$

d_o represents the distance that the point on the manipulator closest to the obstacle advances in the direction of vector $\overrightarrow{C_2C_3}$ after time τ. Designing the obstacle avoidance task reward function:

$$r_o = (1/\,(1 + \exp\,(-k_i \times (D_{GJK} - D_2 - d_o))) - 1) \tag{26}$$

where k_i is a hyperparameter that can adjust the steepness of the function at D_2. The trend of this function is similar to that of the position reward function, with the center point shifted through geometric transformation. Specifically, first, the output of r_o is $(-1, 0)$. When $D_{GJK} < D_1$, a negative reward is always obtained. Second, the center point of the function is shifted along the x-axis by D_2 units. When $D_{GJK} < D_2$, the manipulator quickly receives a large negative reward, prompting it to leave the D_2 region.

4.3. Design of the composite reward function

The reward function is crucial in the DRL training process, and an excellent reward function will significantly affect the convergence speed and training results of the algorithm. Therefore, this paper proposes a composite reward function composed of four parts. First, a position reward function r_p is needed to drive the end-effector of the manipulator to approach the target point. Second, using the GJK distance algorithm, the real-time minimum distance between the manipulator and the obstacle is obtained to calculate the obstacle avoidance reward function r_o, preventing collisions. Finally, to shorten the training time and improve the convergence rate, a time reward function r_t and a reward function r_g after reaching the target point are set, respectively. In summary, the composite reward function is expressed as

$$r = r_p + \eta\,(r_o + r_t) + r_g \tag{27}$$

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993

where r_p is Eq. (22), r_o is Eq. (26), and r_g is a normal constant, usually one order of magnitude higher than the other three parts. r_t is expressed as

$$r_t = k_\alpha \tau \tag{28}$$

where $k_\alpha \in R$ and $k_\alpha < 0$, that is, the time reward r_t at each step is always less than 0. This encourages the manipulator to quickly approach the target point. η is the decay factor, acting on the obstacle avoidance reward function and the time reward function, and is composed of the real-time distance $d_e = \|\overrightarrow{C_0C_1}\|$ between the end-effector of the manipulator and the target point:

$$\eta = k_\lambda \, (d_e / d_{eo}) \tag{29}$$

where d_{eo} is the initial distance between the end-effector and the target point. $k_\lambda > 0$ is the rate at which η changes according to the real-time distance. As the manipulator approaches the target point, the obstacle reward function r_o and the time reward function r_t are constantly less than 0, and the real-time distance $d_e = \|\overrightarrow{C_0C_1}\|$ decreases. The decay factor η can be derived from (29) to decrease with the reduction of d_e, thus reducing the negative reward part $\eta(r_o + r_t)$ in Eq. (27). Therefore, during the process of the manipulator approaching the obstacle, the reward containing the decay factor η is greater than the reward without it.

In DRL, rewards serve to enable agents to distinguish between the quality of actions, thus requiring a certain degree of discriminability. First, the reward function proposed in this paper normalizes rewards using the $f_x = 1/(1 + \exp(x))$ function, which contributes to the stability of the training process. Second, a relatively large number of hyperparameters are utilized; however, these hyperparameters only affect the relative magnitude of rewards without altering their positive or negative nature and consequently do not impact the discriminability of rewards. Modifying these parameters merely influences the direction and speed of the agent's optimization, rather than the training outcomes. In summary, the configuration of the reward function in this paper exhibits favorable generalization ability.

5. Simulation analysis and physical verification

In this section, we will compare the proposed algorithm with other baseline algorithms. To compare the convergence efficiency of the algorithms, the proposed algorithm in this paper is the DGA_TD3 algorithm, which integrates the GJK distance algorithm and the TD3 algorithm. As comparison baselines, we select the DGA_DDPG algorithm, which integrates the GJK distance algorithm and the DDPG algorithm, and the COR_DDPG algorithm, which integrates the artificial potential field method and the DDPG algorithm. To verify the effect of the decay factor η, a baseline algorithm GA_TD3 is set up, which integrates the GJK distance algorithm and the TD3 algorithm but does not include the decay factor. To demonstrate the differences in obstacle avoidance and motion planning between the GJK algorithm and the artificial potential field method, the COR_TD3 baseline algorithm is set up. For the motion planning scheme of the manipulator, in order to verify the performance of the DRL algorithm proposed in this paper with other methods, the meta-heuristic-based RRN algorithm (BAORNN) in ref. [6] is selected as the baseline algorithm. To verify the importance of using joint angular accelerations as the output, a baseline algorithm that uses joint angles as the output is set up. To fully compare the various performance aspects of the proposed algorithm with the baselines, simulations and physical experiments were conducted on a 7-DOF manipulator using the above algorithms.

Specifically, the simulation model of the proposed algorithm was established in MATLAB, and a simulation environment was set up. In the real world, the Robot Operating System (ROS) was used to establish communication with the 7-DOF Sawyer manipulator, creating a practical motion environment. The operating system used for training the algorithm is Windows 11, with an i5-12500H CPU and an NVIDIA RTX 4060 Ti GPU. In the experiment, the end-effector of the 7-DOF manipulator needs to move from the initial position to the target position while avoiding obstacles. Table I lists the other parameter settings used in this paper.

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993

Table I. *Parameter.*

Parameter	Value
Episode	1500
Steps in one episode	150
Discount factor γ	0.99
Learning rate α/β	5e−4/5e−4
Smoothing factor ϕ	5e−2
single-step time $\tau(s)$	5e−1
Rate of change k_p	8e−1
Rate of change k_i	3e−1
Rate of change k_α	5e−1
Rate of change k_λ	5e−1
Joint limits μ, κ_p, κ_v	0.9/0.2/0.2

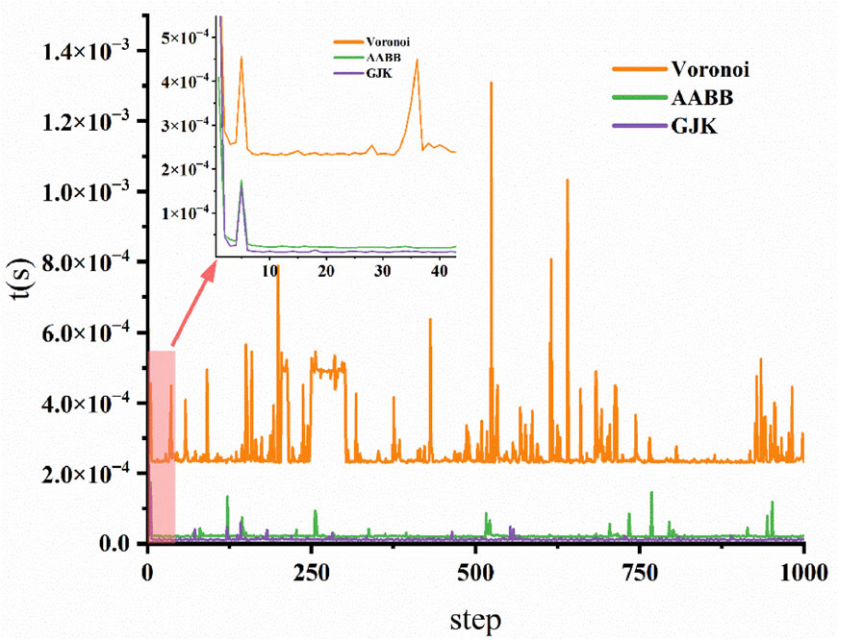


Figure 6. *Efficiency of distance algorithms.*

5.1. Efficiency of distance algorithms

The computational load of the distance algorithm GJK depends on the number of vertices in the 3D models of the two objects, requiring a total of $N_a + N_b$ operations, where N_a and N_b are the number of vertices in the 3D models of the two objects, respectively. For the case of calculating the distance between the manipulator and obstacles, the total number of operations $N = \sum_{i=1}^{m} n_i$ can be obtained, where n_i is the number of operations g between the i-th link of the manipulator and the obstacle, and m is the number of links. If there are k obstacles in the environment, the total number of operations is $N_k = kN$. However, this operation only exists in the first iteration of the algorithm, and the subsequent iteration time is an extremely small constant, as shown in refs. [6, 30, 31]. As shown in Figure 6, we tested the time consumed by the GJK algorithm, AABB algorithm, and Voronoi algorithm to calculate the distance between a single link of the manipulator and an obstacle on a computer. The experiment was conducted 1000 times, and the obstacle contained 32 randomly generated vertices. It can be seen from Figure 6 that the GJK algorithm performs excellently among the three algorithms, with relatively stable computational efficiency, and the calculation time for a single link is controlled at the microsecond level.

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993

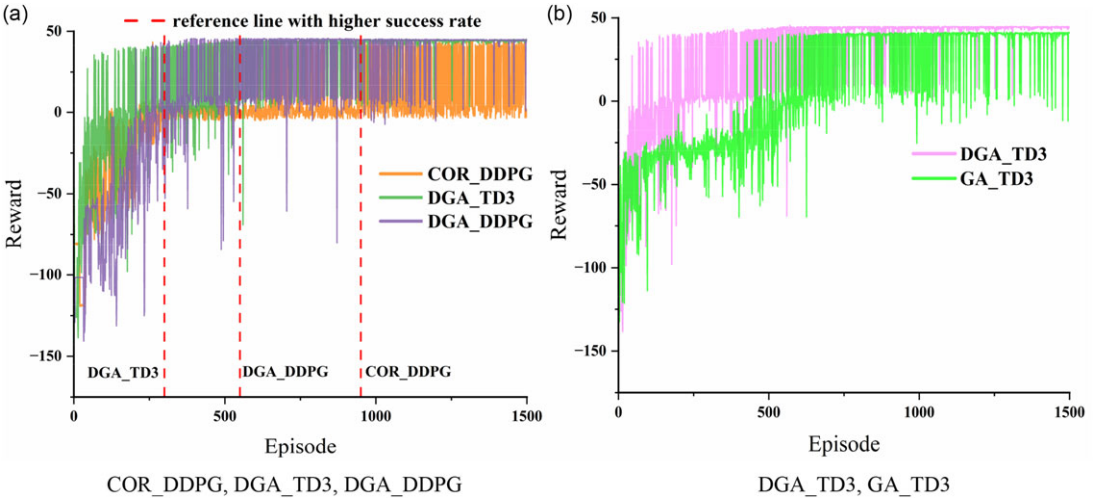


Figure 7. *Reward changes during algorithm training.*

Even if there are multiple obstacles in the environment, the calculation time is at the millisecond level, which can fully meet the requirements of motion control.

5.2. Network training results

The results after 1500 training epochs for each algorithm are shown in Figure 7. Figure 7(a) shows the training results for the COR_DDPG, DGA_DDPG, and DGA_TD3 algorithms, where the red dashed line indicates the reference line for the increased probability of the algorithm reaching the target position. It can be observed that the DGA_TD3 algorithm significantly outperforms the other two algorithms. Specifically, it has a lower probability of reaching the target position in the early stages of training and achieves the maximum reward value. Additionally, it can stably reach the target position in the middle and later stages of training until it fully converges. Comparing the COR_DDPG and DGA_DDPG algorithms, it is evident that DGA_DDPG outperforms COR_DDPG. COR_DDPG does not converge even at the end of training, whereas DGA_DDPG, although performing poorly in the early stages of training, can converge in the later stages. Figure 7(b) compares the training results of the DGA_TD3 and GA_TD3 algorithms. The curve of the GA_TD3 algorithm indicates that the lack of a decay factor may lead to local optima in the early stages of training. However, in the middle and later stages, it can escape these local optima and successfully reach the target position more stably.

To provide a more intuitive and clear comparison of the four algorithms, Figures 8 and 9 are supplementary data for Figure 7. Figure 8(a) compares the success rates of the four algorithms over 1500 training epochs. It can be seen from Figure 8(a) that DGA_TD3 has the highest success rate, followed by GA_TD3 and DGA_DDPG, while COR_DDPG has the lowest success rate. In Figure 9, further analysis of the success rates at different stages of the entire training episode can indirectly reflect the convergence speed and convergence ability of the algorithms. DGA_TD3 still leads the other algorithms at different stages, with the trends of the other algorithms being similar to those in Figure 8(a). Figure 8(b) shows the average number of steps per episode for the four algorithms during training. Since the total number of steps per episode is 150, but an episode will end early if the target point is reached, the average number of steps can also reflect the performance of the algorithms. In summary, the performance of the DGA_TD3 algorithm is far superior to the other three algorithms. However, since the training effect of the TD3 algorithm is better than that of the DDPG algorithm, the training effect of the GA_TD3 algorithm, which lacks a decay factor, is somewhat reduced, resulting in similar performance between the GA_TD3 and DGA_DDPG algorithms. The COR_DDPG algorithm has the lowest performance and cannot fully converge. This is because the artificial potential field method is not suitable for scenes with irregular obstacles, and the algorithm itself has certain defects, making it difficult to obtain the correct solution.

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993

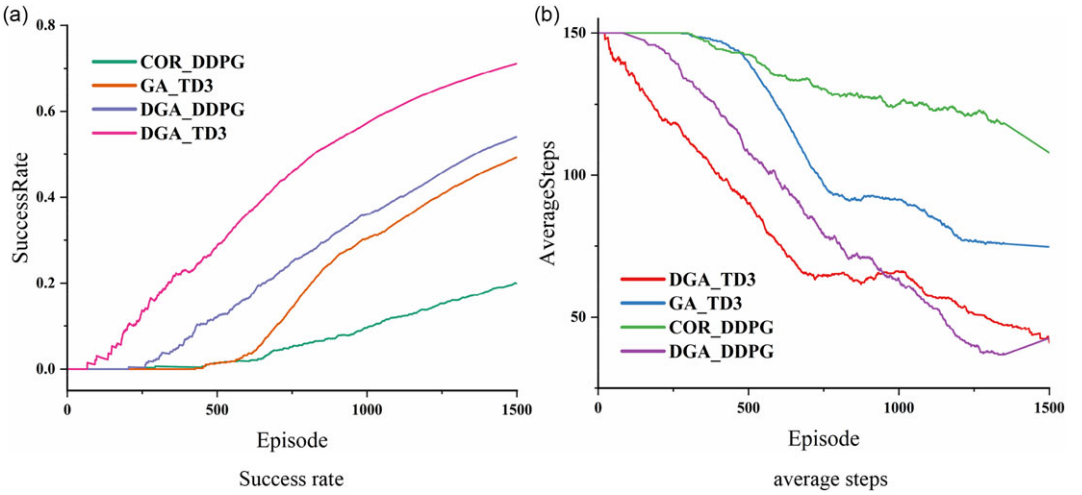


Figure 8. *Success rate and average steps during algorithm training.*

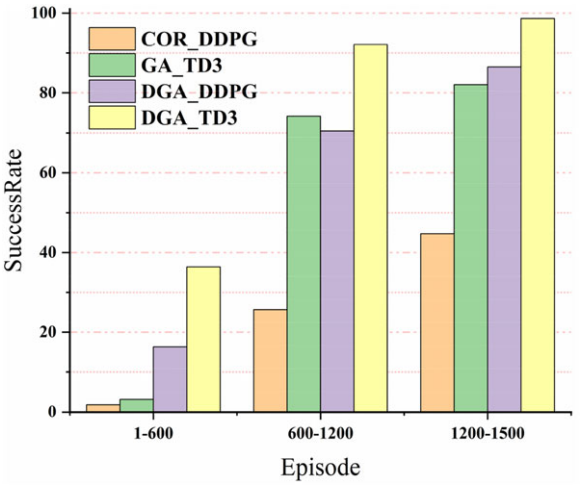


Figure 9. *Comparison of success rates at different stages.*

5.3. Motion planning and obstacle avoidance analysis

Set the safety threshold distance $D_1 = 8$ cm and $D_2 = 4$ cm. In this section, the differences in motion paths among the DRL algorithms using the artificial potential field method (COR_TD3) [28], the GJK method (DGA_TD3), and the BAORNN algorithm using the GJK algorithm [6] will be compared. Figures 10 and 11 show the motion planning results of the GJK algorithm and the artificial potential field algorithm, respectively, and Figure 12 presents the motion planning results of the BAORNN algorithm. From the analysis of the motion results in Figure 11, since the artificial potential field method will expand the range of obstacles, the path using the artificial potential field algorithm usually moves away from the obstacles first to obtain a larger reward. However, because the distance between the target position set in the training process and the obstacle is relatively close, the manipulator will be attracted to the target position. As a result, the position of the manipulator's links is not optimal and may be very close to the obstacle. This is also one of the direct reasons for the poor training effect in the previous section, because contact with the obstacle during training is directly judged as a failure.

From the analysis of the motion results in Figure 12, the motion results of the BAORNN algorithm using the GJK method are better than those of the COR_TD3 algorithm. This algorithm ensures that

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993

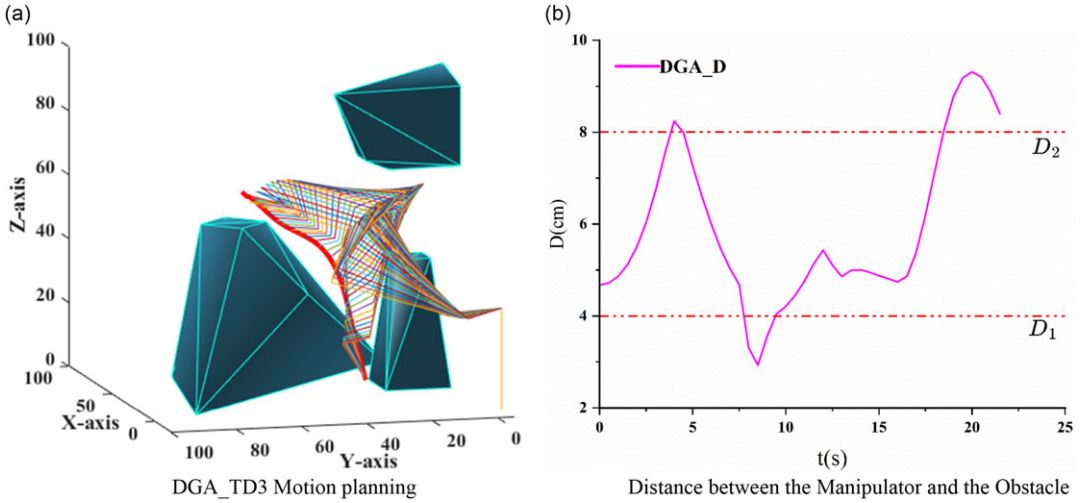


Figure 10. *Motion planning and obstacle avoidance with DGA_TD3.*

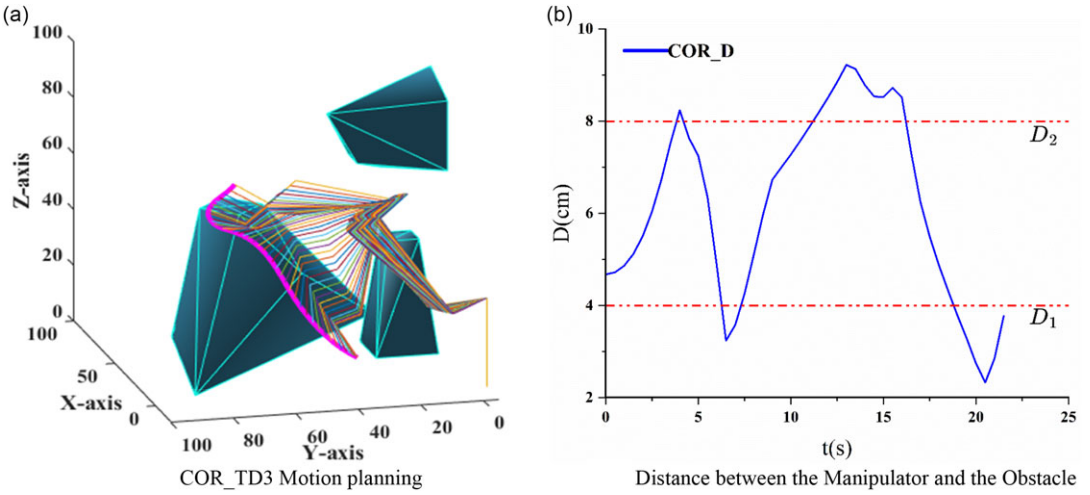


Figure 11. *Motion planning and obstacle avoidance with COR_TD3* [28].

the manipulator does not collide with obstacles, and at the same time, keeps the end effector of the manipulator from being too far away from the target point. However, because the output of this algorithm is the joint position and the step size needs to be adjusted manually, the planned trajectory is not very smooth, and there are multiple cases where the end effector suddenly changes its movement direction. In practical manipulator applications, this will cause abrupt changes in the torque or acceleration of the manipulator joints, which will damage the joints of the manipulator to a certain extent. Comparing the motion planning results in Figure 10, the DGA_TD3 algorithm can plan an efficient and smooth motion trajectory on the premise of ensuring the safety of the manipulator. To sum up, the method proposed in this paper, which integrates the GJK distance algorithm into reinforcement learning, performs better when facing multiple irregular obstacles in the environment.

Figures 13 and 14 show the change curves of the joint information of the manipulator during the motion planning process. Figure 13 is the joint angle as the algorithm output, and Figure 14 is the joint angular acceleration as the algorithm output, both of which use the same TD3 algorithm framework. Considering the real application scenarios of the manipulator, it is necessary to limit the joint information for both algorithms. For the algorithm that uses joint angles as output, the joint angle limit is

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993

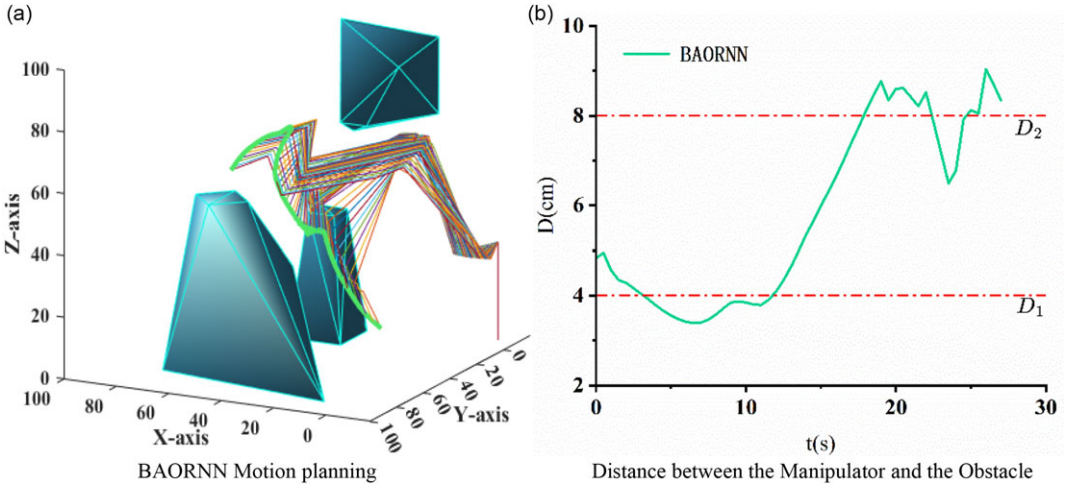


Figure 12. *Motion planning and obstacle avoidance with BAORNN* [6].

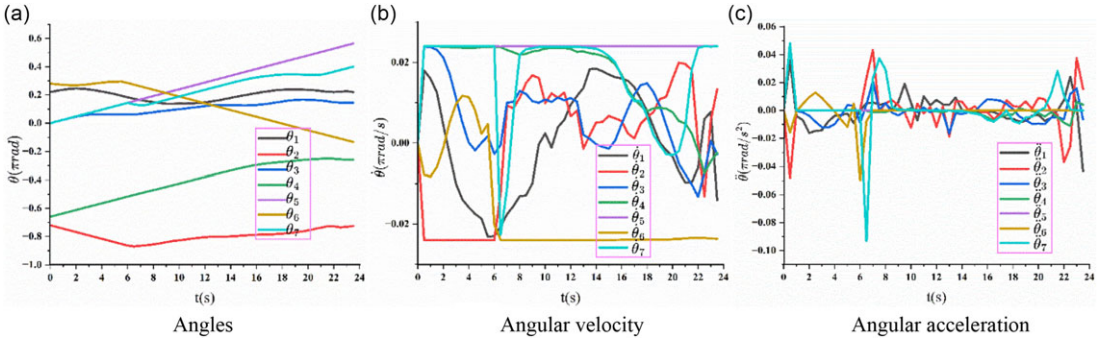


Figure 13. *Algorithm with joint angles output.*

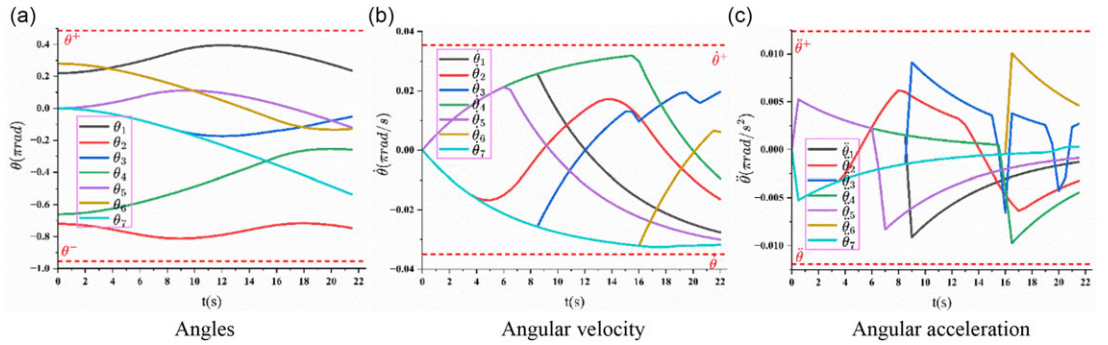


Figure 14. *Algorithm with joint angular acceleration output.*

set to $\pm 0.85\pi$ rad, and the joint angular velocity limit is set to $\pm 0.024\pi$ rad s^{-1}. For the algorithm that uses joint angular accelerations as output, the joint physical limit mechanism mentioned in this paper is used, with the joint angle limit set to $\pm 0.85\pi$ rad, the joint angular velocity limit set to $\pm 0.035\pi\, rad/s$, and the joint angular acceleration limit set to $\pm 0.0015\pi$ rad s^{-2}. In Figures 13(a) and 14(a), it can be seen that the algorithm that uses joint angular accelerations as output has smoother joint angle changes, better meeting the needs of real manipulators. The essence of this phenomenon is whether the torque changes of each joint of the manipulator are stable, i.e., the changes in joint angular acceleration. From

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993

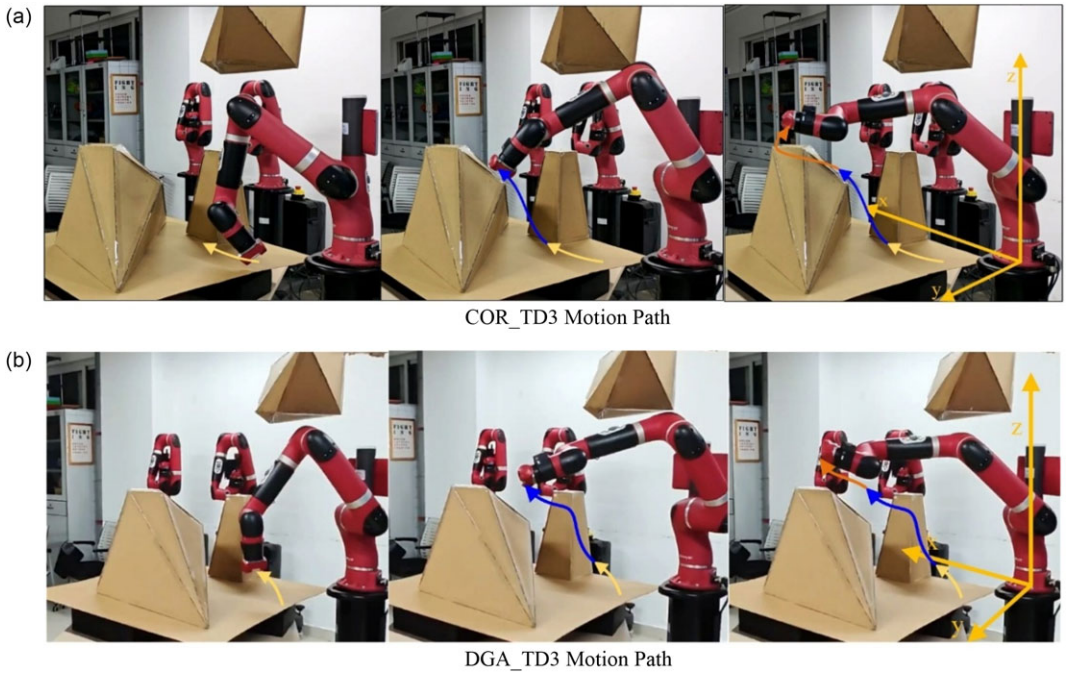


Figure 15. *Motion planning for manipulators in real environments.*

Figure 13(c), it is analyzed that the algorithm with joints as outputs produces an abrupt change in angular acceleration during motion, it reaches a large peak, e.g., $\ddot{\theta}_7 = -0.093\pi$ rad s^{-1} when $t = 6.5s$, which will lead to abrupt changes in the output torque. If such situations frequently occur during motion, the manipulator is very likely to be damaged. However, in Figure 14(c), the algorithm that uses joint angular accelerations as output always ensures that the peak value is within a controllable range. Therefore, the latter can greatly reduce the probability of manipulator damage while completing the task. This proves the reliability and advancement of the method used in this paper using joint angular acceleration as the output.

5.4. Real environment validation

In this section, we will validate the proposed method in this paper using a real manipulator in the real world. The algorithm validation section can be divided into the following points: First, a MATLAB script is used to complete the training of the DRL model in the simulation environment. Second, the kinematics, dynamics model, control law, collision detection algorithm, and other components of the manipulator are encapsulated into an SDK software package and deployed on the Sawyer manipulator. Third, a communication network node is constructed using the ROS, with the network protocol being the TCP/IP protocol, forming a distributed communication network. This network enables real-time communication between the computer and the manipulator, including the computer publishing and subscribing to messages that the manipulator receives to control joint movements; and the manipulator feeding back observational information, allowing the computer to obtain the manipulator's state and output the next expected action through the trained reinforcement learning model. The system is composed of a remote computer, a local computer of the manipulator, a software operating system (Ubuntu 20), and the ROS1 operating system.

Figure 15 shows the motion planning tasks completed by the manipulator driven by the baseline algorithm COR_TD3 (Figure 15(a)) and the proposed DGA_TD3 algorithm (Figure 15(b)), with the motion paths of the manipulator labeled at the corresponding stages. Three-dimensional coordinate differences between the end-effector of the manipulator and the target position were recorded during the

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993

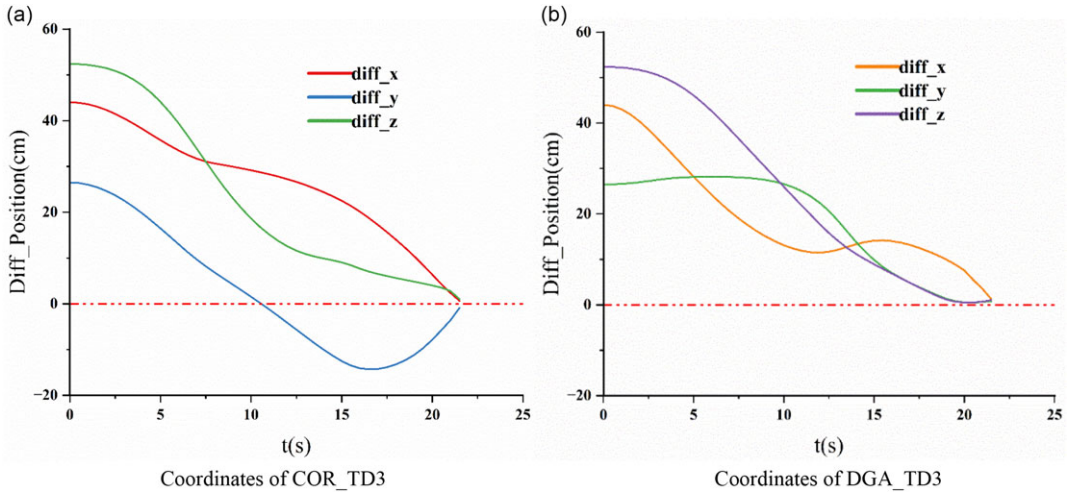


Figure 16. *Difference between manipulator and target coordinates.*

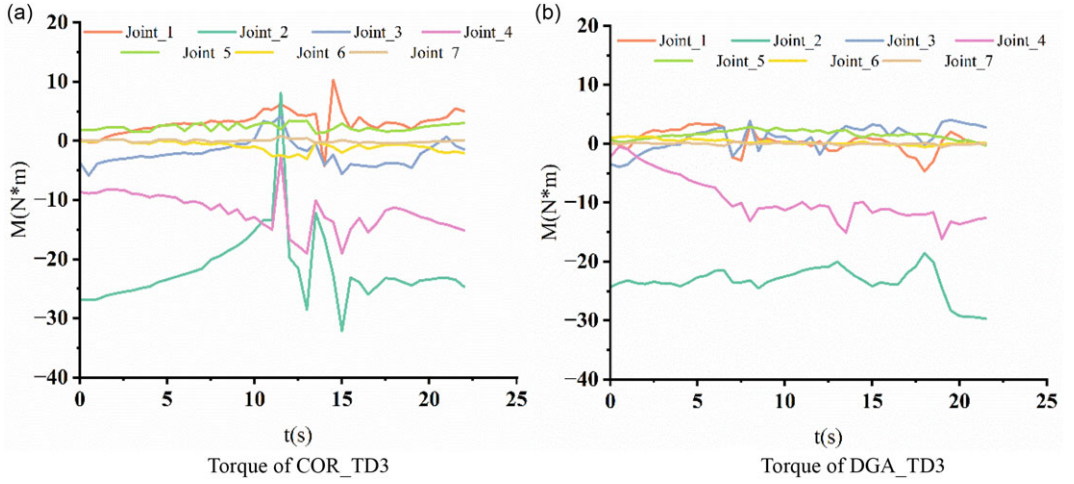


Figure 17. *Changes in joint torque.*

motion, as shown in Figure 16. Comparing Figures 16(a) and (b), it is obvious to conclude that the trajectory in Figure 16(a) (COR_TD3) is worse than the trajectory in Figure 16(b) (DGA_TD3). DGA_TD3 can clearly reach the target point more efficiently, while the *y*-coordinate of COR_TD3 exceeds the given target *y*-value during the motion, indicating that the manipulator made an unnecessary decision to move away from the target point during the motion. Figure 17 depicts the torque variations of the seven joints during the motion of the manipulator. Specifically, Figure 17(a) illustrates the torque output characteristics of the baseline algorithm COR_TD3 for angle output, which exhibits distinct short-term abrupt changes in torque. In contrast, Figure 17(b) shows that the DGA_TD3 algorithm proposed in this study maintains stable torque throughout the entire motion process, with no significant abrupt changes observed.

Such a difference holds significant implications in practical applications. Taking the Sawyer manipulator used in this study as an example, which is driven by motorized joints, abrupt changes in torque would imply that a large current flows through the motor in a short period. This not only may compromise the operational stability of the manipulator but also pose a risk of causing permanent damage to the joints. Therefore, to summarize, in the real world, the method proposed in this paper can also exhibit better performance than the baseline algorithm.

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993

6. Conclusion

This paper proposes a motion planning scheme for redundant manipulators to avoid obstacles based on a DRL algorithm, considering the actual shapes of obstacles. Starting from the kinematics model of the manipulator, the scheme uses the GJK distance algorithm to obtain the actual distance from obstacles. Based on this distance, a composite reward function for DRL is constructed to complete the obstacle avoidance, path planning, and trajectory planning tasks during the manipulator's motion. The GJK distance algorithm provides a more accurate and reliable distance from obstacles. The action output of the DRL algorithm is the joint angular acceleration, which avoids complex IK calculations and sudden changes in joint velocity during obstacle avoidance. The designed composite reward function improves the training efficiency of the algorithm and accelerates its convergence time, with a more significant improvement after the introduction of the decay factor. Finally, the feasibility of the scheme is verified on a 7-DOF manipulator in both simulation and real-world environments. Compared with other baseline algorithms, the scheme demonstrates better strategy selection in path planning and trajectory planning. In future work, we will consider the following two aspects: First, after reaching the target point, the velocities and accelerations of all joints of the manipulator will be reduced to 0, which is necessary for multi-task motion in real engineering applications. Second, the dynamics of the manipulator will be combined with the improved kinematics solution proposed in this paper to achieve a complete control process.

Author contributions. Qing Yang: Conceptualization, writing—review and editing. Ju Chen: Investigation, methodology, writing—original draft, and writing—review and editing. Yi Zhang: Validation, visualization. Liang Ge: Funding acquisition. Yun Xiao Wang: Data curation, visualization.

Financial support. This work was supported by the National Natural Science Foundation of China (No. 52374234).

Competing interests. I have nothing to declare. The authors have no relevant financial or non-financial interests to disclose.

Ethical approval. Not applicable.

References

[1] A. Xie, T. Chen, G. Zhang, Y. Li and X. Rong, "Manipulability enhancement of legged manipulators by adaptive motion distribution," *IEEE Trans. Ind. Electron.* **72**(1), 724–733 (2024). doi: 10.1109/TIE.2024.3413833.

[2] Z. Zhang, X. Liu, M. Ning, X. Li, W. Liu and Y. Lu, "A review of motion planning for redundant space manipulators," *Sci. China Technol. Sci.* **68**, 1310401 (2025). doi: 10.1007/s11431-024-2841-y.

[3] Y. Li, Q. Feng, Y. Zhang, C. Peng, Y. Ma, C. Liu, M. Ru, J. Sun and C. Zhao, "Peduncle collision-free grasping based on deep reinforcement learning for tomato harvesting robot," *Comput. Electron. Agric.* **216**, 108488–1699 (2024). doi: 10.1016/j.compag.2023.108488. ISSN 0168-1699.

[4] W. Yuan, Z. Li and C.-Y. Su, "Multisensor-based navigation and control of a mobile service robot," *IEEE Trans. Syst., Man, Cybern.: Syst.* **51**(4), 2624–2634 (2021). doi: 10.1109/TSMC.2019.2916932.

[5] H. Yang, D. Li, X. Xu and H. Zhang, "An obstacle avoidance and trajectory tracking algorithm for redundant manipulator end," *IEEE Access* **10**, 52912–52921 (2022). doi: 10.1109/ACCESS.2022.3173404.

[6] A. H. Khan, S. Li and X. Luo, "Obstacle avoidance and tracking control of redundant robotic manipulator: An RNN-based metaheuristic approach," *IEEE Trans. Ind. Inform.* **16**(7), 4670–4680 (2020). doi: 10.1109/TII.2019.2941916.

[7] Y. Cong, H. Du, W. Chen, W. Zhu and G. Wen, "A Lyapunov-based step-by-step sliding-mode observer algorithm with application to joint torque estimation of robot manipulators," *IEEE Trans. Ind. Inform.* **21**(3), 2451–2460 (2024). doi: 10.1109/TII.2024.3507208.

[8] D. S. Guo and Y. N. Zhang, "Different-level two-norm and infinity-norm minimization to remedy joint-torque instability/divergence for redundant robot manipulators," *Robot. Auton. Syst.* **60**(6), 874–888 (2012). doi: 10.1016/j.robot.2012.01.008. ISSN. 0921-8890.

[9] B. Feng, X. Jiang, B. Li, Q. Zhou, Y. Bi and B. Part, "An adaptive multi-RRT approach for robot motion planning," *Expert Syst. Appl.* **252**, 124281 (2024). doi: 10.1016/j.eswa.2024.124281. ISSN 0957-4174.

[10] J. Zhang, J. Wu and X. Shen, "Autonomous land vehicle path planning algorithm based on improved heuristic function of A-star," *Int. J. Adv. Robot. Syst.* **18**(5), 1729–8814 (2021). doi: 10.1177/17298814211042730.

[11] J. Gammell, T. Barfoot and S. Srinivasa, "Batch informed trees (BIT*): Informed asymptotically optimal anytime search," *Int. J. Robot. Res.* **39**(5), 543–567 (2020). doi: 10.1177/0278364919890396.

[12] A. T. Khan and M. Ijaz, "Bio-inspired BAS: Run-time path-planning and the control of differential mobile robot," *EAI Endors. Trans. AI Robot.* 1–10 (2022). doi: 10.4108/airo.v1i.656.

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993

[13] F. Ju, H. Jin, B. Wang and J. Zhao, "A predictable obstacle avoidance model based on geometric configuration of redundant manipulators for motion planning," *Sensors* **23**(4642), 4642 (2023). doi: 10.3390/s23104642.
[14] Z. G. Mu, W. F. Xu and B. Liang, "Avoidance of multiple moving obstacles during active debris removal using a redundant space manipulator," *Int. J. Control, Autom. Syst.* **15**(2), 815–826 (2017). doi: 10.1007/s12555-015-0455-7.
[15] J. Liu, Y. Tong, Z. Ju and Y. Liu, "Novel method of obstacle avoidance planning for redundant sliding manipulators," *IEEE Access* **8**, 78608–78621 (2020). doi: 10.1109/ACCESS.2020.2990555.
[16] B. Ma, Z. Xie, B. Zhan, Z. Jiang, Y. Liu and H. Liu, "Actual shape-based obstacle avoidance synthesized by velocity–Acceleration minimization for redundant manipulators: An optimization perspective," *IEEE Trans. Syst., Man, Cybern.: Syst.* **53**(10), 6460–6474 (2023). doi: 10.1109/TSMC.2023.3283266.
[17] Y. Peng, B. Tang, D. Huang and Y. Wei, "Manipulator trajectory planning based on clustering curve discretization and B-spline," *J. Field Robot.*, 1556–4959 (2024). doi: 10.1002/rob.22485.
[18] D. Zhang, R. Ju and Z. Cao, "Reinforcement learning-based motion control for snake robots in complex environments," *Robotica* **42**(4), 947–961 (2024). doi: 10.1017/S0263574723001613.
[19] N. Khlif, K. Nahla and B. Safya, "Reinforcement learning with modified exploration strategy for mobile robot path planning," *Robotica* **41**(9), 2688–2702 (2023). doi: 10.1017/S0263574723000607.
[20] Z. Bai, H. Pang, Z. He, B. Zhao and T. Wang, "Path planning of autonomous mobile robot in comprehensive unknown environment using deep reinforcement learning," *IEEE Internet Things* **11**(12), 22153–22166 (2024). doi: 10.1109/JIOT.2024.3379361.
[21] X. Gan, Z. Huo and W. Li, "DP-A*: For path planing of UGV and contactless delivery," *IEEE Trans. Intell. Transp.* **25**(1), 907–919 (2024). doi: 10.1109/TITS.2023.3258186.
[22] F. Ying, H. Liu, R. Jiang and X. Yin, "Trajectory generation for multiprocess robotic tasks based on nested dual-memory deep deterministic policy gradient," *IEEE/ASME Trans. Mechatron.* **27**(6), 4643–4653 (2022). doi: 10.1109/TMECH.2022.3160605.
[23] Y. X. Cao, S. J. Wang, X. Zheng and L. Liu, "Planning and Control of Space Robot in Capture Operation Based on Reinforcement Learning," **In:** *Chinese Control Conference,* Shanghai, July 26–282021).
[24] B. Y. Song, J. Q. Li, X. Y. Liu and G. L. Wang, "A trajectory planning method for capture operation of space robotic arm based on deep reinforcement learning," *J. Comput. Inf. Sci. Eng.* **24**(9), 091003 (2024). doi: 10.1115/1.4065814.
[25] S. De Witte, T. Van Hauwermeiren, T. Lefebvre and G. Crevecoeur, "Learning to cooperate: A hierarchical cooperative dual robot arm approach for underactuated pick-and-placing," *IEEE/ASME Trans. Mechatron.* **27**(4), 1964–1972 (2022). doi: 10.1109/TMECH.2022.3175484.
[26] Z. Sui, Z. Pu, J. Yi and S. Wu, "Formation control with collision avoidance through deep reinforcement learning using model-guided demonstration," *IEEE Trans. Neur. Netw. Learn.* **32**(6), 2358–2372 (2021). doi: 10.1109/TNNLS.2020.3004893.
[27] J. Dai, Y. Zhang and H. Deng, "Novel potential guided bidirectional RRT∗ with direct connection strategy for path planning of redundant robot manipulators in joint space," *IEEE Trans. Ind. Electron.* **71**(3), 2737–2747 (2024). doi: 10.1109/TIE.2023.3269462.
[28] H. Liu, F. Ying, R. Jiang, Y. Shan and B. Shen, "Obstacle-avoidable robotic motion planning framework based on deep reinforcement learning," *IEEE/ASME Trans. Mechatron.* **29**(6), 4377–4388 (2024). doi: 10.1109/TMECH.2024.3377002.
[29] Y. N. Zhang, D. S. Guo and S. G. Ma, "Different-level simultaneous minimization of joint-velocity and joint-torque for redundant robot manipulators," *J. Intell. Robot. Syst.: Theory Appl.* **72**(3-4), 301–323 (2013). doi: 10.1007/s10846-013-9816-8
[30] E. G. Gilbert, D. W. Johnson and S. S. Keerthi, "A fast procedure for computing the distance between complex objects in three-dimensional space," *IEEE J. Robot. Autom.* **4**(2), 193–203 (1988). doi: 10.1109/56.2083.
[31] M. Montanari, N. Petrinic and E. Barbieri, "Improving the GJK algorithm for faster and more reliable distance queries between convex objects," *ACM Trans. Graph* **36**(3), 30 (2017). doi: 10.1145/3083724.
[32] C. Wang, C. G. Frazelle, J. R. Wagner and I. D. Walker, "Dynamic control of multisection three-dimensional continuum manipulators based on virtual discrete-jointed robot models," *IEEE/ASME Trans. Mechatron.* **26**(2), 777–788 (2021). doi: 10.1109/TMECH.2020.2999847.

Cite this article: Q. Yang, J. Chen, Y. Zhang, L. Ge and Y. Wang (2025). "Deep reinforcement learning-based obstacle avoidance motion planning for redundant manipulator considering the actual shape of obstacles", Robotica **43**, 4519–4537. https://doi.org/10.1017/S0263574725102993

Downloaded from https://www.cambridge.org/core. Shanghai University Library, on 09 Oct 2026 at 01:46:29, subject to the Cambridge Core terms of use, available at https://www.cambridge.org/core/terms. https://doi.org/10.1017/S0263574725102993